

TEMA 3 → ARQUITECTURA WEB Y FUNDAMENTOS DEL DESPLIEGUE

3.1 Introducción al Despliegue de Aplicaciones Web

El **desarrollo web moderno** es un ámbito en **constante evolución** que abarca la **creación y gestión** de sitios web y aplicaciones que funcionan a través de Internet.

Los objetivos clave del despliegue son garantizar accesibilidad, estabilidad, escalabilidad y seguridad de las aplicaciones. Un despliegue eficiente ofrece estos beneficios principales:

- **Acelera el Time-to-Market** para lanzar productos con mayor agilidad
- **Permite iteración constante** y entrega continua de mejoras
- **Reduce errores humanos** mediante automatización
- **Libera a los equipos** para tareas más estratégicas
- **Aumenta la competitividad** mediante servicios confiables y de calidad
- **Facilita la documentación** para reproducir procesos, resolver problemas y capacitar nuevos miembros

3.2 Front-end vs Back-end

El **Front-end** es la parte visible con la que el usuario interactúa directamente en el navegador. Se encarga del diseño, la estructura visual y la experiencia de usuario, utilizando tecnologías como HTML, CSS, JavaScript y frameworks como React o Angular. Sus preocupaciones incluyen la usabilidad, la accesibilidad y el SEO.

El **Back-end** es la parte que se ejecuta en el servidor y gestiona la lógica interna de la aplicación: procesar peticiones, acceder a bases de datos, manejar la seguridad y comunicarse con el Front-end. Utiliza lenguajes como PHP, Python, Java o Node.js.

Actualmente, el Back-end suele ser universal o agnóstico gracias al uso de **APIs**, que permiten intercambiar datos en formatos como JSON o XML. Esto posibilita que un mismo Back-end sirva a distintos clientes (web, móviles, etc.), facilitando la integración, la escalabilidad y la reutilización del código.

3.3 Web vs App Web

Aunque a veces se confunden, una página web y una aplicación web no son lo mismo.

Una **página web** es principalmente informativa. Puede ser estática (su contenido es fijo a menos que se edite manualmente) o dinámica (su contenido se genera desde un servidor). Las páginas estáticas pueden verse sin un servidor, mientras que las dinámicas sí lo requieren.

Una **aplicación web** es un sistema más complejo que ofrece servicios o funcionalidades interactivas. Aunque se ejecuta en un navegador, funciona como una aplicación de escritorio, permitiendo registrar usuarios, gestionar datos y realizar tareas específicas. Siempre necesita un servidor web y, normalmente, una base de datos.

3.3 Arquitecturas web y su impacto en el despliegue

La arquitectura condiciona cómo empaquetar y desplegar la aplicación.

Arquitectura Monolítica

- **Qué hace:** Agrupa interfaz, lógica y datos en una sola aplicación.
- **Despliegue:** Se empaqueta como una unidad (WAR/JAR, imagen Docker) y se ejecuta como un proceso único.
- **Ideal para:** Proyectos pequeños y MVPs.

Arquitectura de Capas (MVC)

- **Qué hace:** Separa responsabilidades en Modelo, Vista y Controlador.
- **Despliegue:** Se publica como aplicación única pero con componentes separados en la infraestructura.
- **Ideal para:** Aplicaciones empresariales con requisitos claros.

Arquitectura de Microservicios

- **Qué hace:** Divide la aplicación en servicios pequeños e independientes.
- **Despliegue:** Cada servicio se empaqueta y despliega por separado usando orquestadores como Kubernetes.
- **Ideal para:** Aplicaciones complejas y grandes empresas.

Arquitectura Serverless (FaaS)

- **Qué hace:** Implementa funciones que se ejecutan bajo demanda por eventos.
- **Despliegue:** Se suben funciones al proveedor cloud con triggers configurados.
- **Ideal para:** Funciones esporádicas y tareas basadas en eventos.

Arquitectura SOA/EDA

- **Qué hace:** Servicios independientes se comunican mediante mensajes o eventos.
- **Despliegue:** Requiere despliegue de servicios y configuración de un broker de eventos.
- **Ideal para:** Integración entre sistemas y procesamiento asíncrono.

Cada arquitectura determina cómo se empaqueta, despliega y escala la aplicación, siendo crucial elegir la adecuada según la complejidad del proyecto.

3.4 Protocolos y comunicación. HTTP

El Protocolo HTTP es la base de la comunicación web. Es un protocolo sin estado y no orientado a la conexión, donde cada intercambio entre cliente y servidor es independiente.

Características principales:

- **Sencillo:** Basado en texto plano, fácil de entender.
- **Extensible:** Permite añadir cabeceras para información adicional.
- **Sin estado:** Cada solicitud se procesa de manera aislada (se usan cookies y sesiones para mantener estado).
- Es fundamental para microservicios y APIs REST.

Ventajas:

Control eficiente de caché, autenticación de usuarios, uso transparente de proxies y manejo de estado mediante sesiones.

Formato de Peticiones y Respuestas HTTP

- **Petición:** Incluye método (GET), ruta (/index.html) y versión del protocolo, seguido de cabeceras.
- **Respuesta:** Contiene versión del protocolo, código de estado (200 OK) y descripción, seguido del contenido solicitado después de una línea vacía.

`POST /?id=1 HTTP/1.1` **Request line**

```
Host: www.swingvy.com
Content-Type: application/json; charset=utf-8
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.12; rv:53.0)
Gecko/20100101 Firefox/53.0
Connection: close
Content-Length: 136
```

Header

```
{
  "status": "ok",
  "extended": true,
  "results": [
    {"value": 0, "type": "int64"},
    {"value": 1.0e+3, "type": "decimal"}
  ]
}
```

Body message

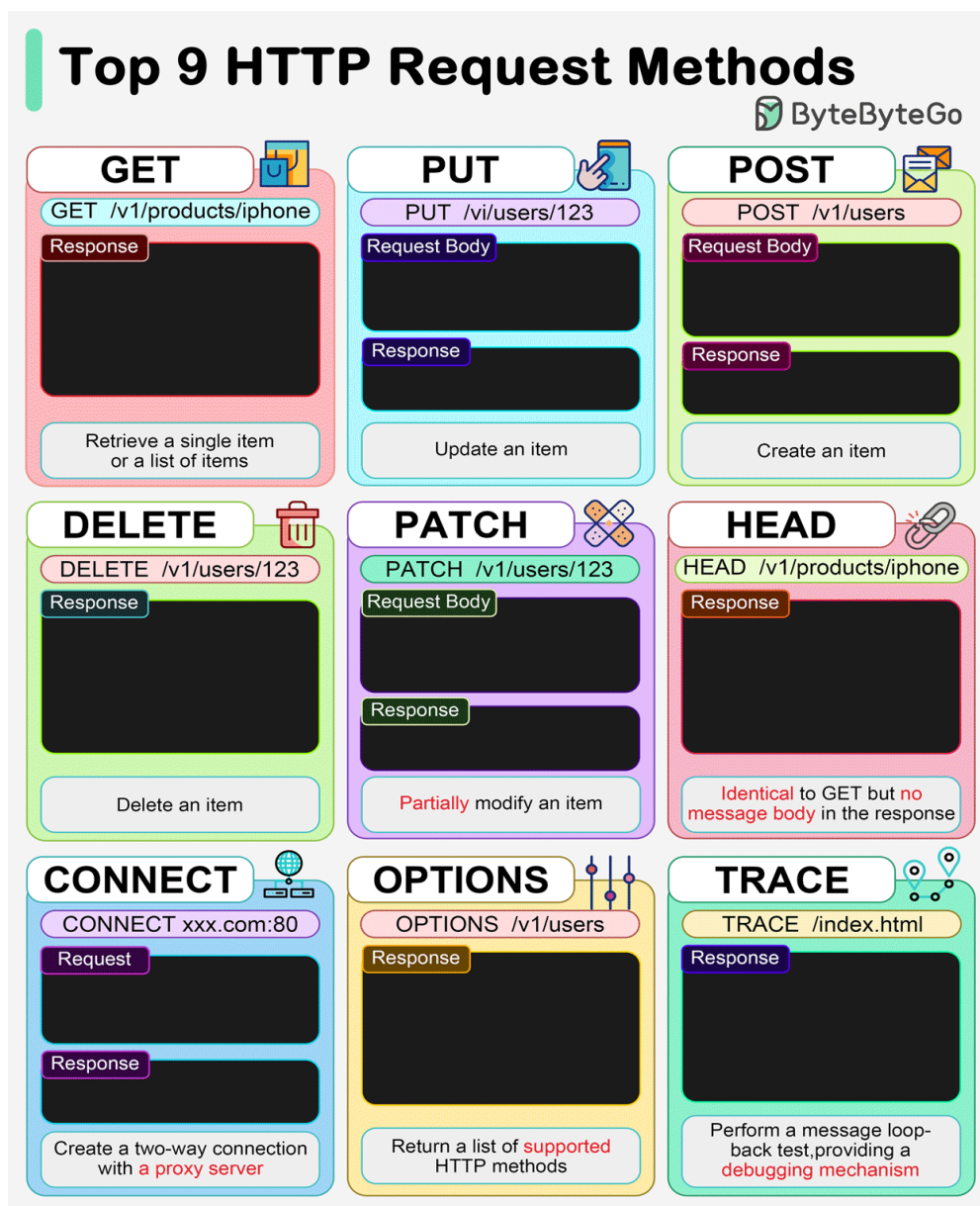
Cabeceras HTTP

Las **cabeceras HTTP** son elementos adicionales que acompañan tanto a las peticiones como a las respuestas y aportan información esencial sobre la comunicación.

Métodos o Verbos HTTP (GET, POST, PUT, DELETE, HEAD)

Los **métodos HTTP** —también conocidos como **verbos**— determinan la acción que el cliente desea ejecutar sobre un recurso del servidor.

- **GET**: Se emplea para **obtener** o consultar un recurso. Generalmente no incluye cuerpo en la solicitud; los parámetros se añaden a la URL como *query string*.
- **POST**: Se utiliza para **enviar** datos al servidor o **crear** un nuevo recurso. Los datos se incluyen en el cuerpo de la petición y no son visibles en la URL.
- **PUT**: Permite **actualizar** o **reemplazar** por completo un recurso existente.
- **DELETE**: Se usa para **eliminar** un recurso determinado del servidor.
- **HEAD**: Solicita únicamente las cabeceras que devolvería un GET, sin el cuerpo. Es útil para comprobar la existencia o los metadatos de un recurso sin descargarlo.



Códigos de Estado HTTP

Cada respuesta del servidor incluye un **código de estado HTTP**, un número de tres cifras que comunica el resultado de la solicitud.

- **1XX (Informativas)**: La petición ha sido recibida y se continúa procesando.
- **2XX (Éxito)**: La solicitud fue recibida, comprendida y procesada correctamente. Ejemplo: **200 OK**.
- **3XX (Redirección)**: Se requiere una acción adicional por parte del cliente (por ejemplo, el recurso ha cambiado de ubicación).
- **4XX (Error del Cliente)**: Indica un problema en la petición (por ejemplo, **403 Forbidden**, **404 Not Found**).
- **5XX (Error del Servidor)**: Señala un fallo interno al intentar procesar una solicitud válida.

HTTPS (SSL/TLS y Certificados Digitales):

HTTPS es la versión segura de HTTP, esencial para proteger la comunicación confidencial entre cliente y servidor. A diferencia de HTTP que transmite datos en texto plano, HTTPS cifra toda la comunicación para garantizar la privacidad.

Base de seguridad:

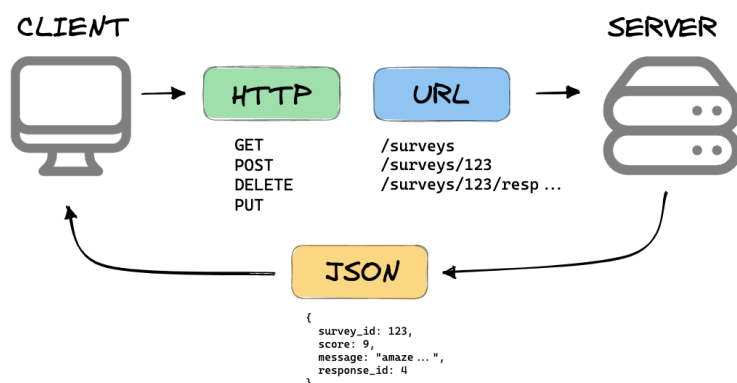
- Utiliza **certificados digitales** que asocian una clave pública con la identidad del propietario (como un sitio web)
- Son emitidos por **Autoridades de Certificación (AC)** que los firman digitalmente para validar su autenticidad

Verificación:

- Los navegadores verifican automáticamente la validez de los certificados
- Alertan al usuario cuando un certificado es inválido, autofirmado o no coincide con el dominio
- Muestran advertencias de seguridad en caso de problemas con el certificado

HTTPS protege contra la interceptación de datos y garantiza que los usuarios se comuniquen con el servidor legítimo.

WHAT IS A REST API?



3.5 Servicio Web y Comunicación con APIs

3.5.1 ¿Qué es un Servicio Web?

Un servicio web es un sistema que permite que dos aplicaciones se comuniquen a través de la web para intercambiar datos o utilizar funcionalidades remotas. Se apoya en protocolos estándar (típicamente HTTP) y en formatos de datos como JSON o XML.

Beneficios principales:

- Integración: conecta sistemas heterogéneos.
- Escalabilidad y flexibilidad: divide funcionalidades en servicios reutilizables.
- Reutilización: un mismo servicio puede consumirse desde web, móvil o escritorio.
- Independencia del cliente: el servidor entrega datos, no vistas.

3.5.2 Protocolos de Comunicación para APIs

La elección del protocolo o estilo de comunicación de la API es una decisión arquitectónica que afecta rendimiento, escalabilidad, experiencia de usuario y costes. A continuación, se presenta una comparación de los protocolos más usados.

Comparativa: REST, GraphQL, gRPC, WebSocket, SOAP

REST:

¿Qué es?

Arquitectura que usa URLs y verbos HTTP (GET, POST, etc.) para manipular recursos.

Simple y estándar, ideal para CRUD y APIs públicas

GraphQL:

¿Qué es?

Lenguaje de consulta donde el cliente define qué datos quiere recibir.

Flexible, cliente elige datos, perfecto para apps complejas.

gRPC:

¿Qué es?

Comunicación entre servicios usando contratos fuertemente tipados y binarios.

Máximo rendimiento, mejor para comunicación interna entre servicios

WebSocket :

¿Qué es?

Conexión persistente para comunicación bidireccional en tiempo real.

Tiempo real bidireccional, ideal para chats y actualizaciones live.

SOAP:

Protocolo empresarial basado en XML con estándares estrictos y WSDL.

Empresarial y seguro, usado en sistemas legacy corporativos.

3.6 Tecnologías para el Desarrollo de Servicios

El desarrollo de servicios se centra en la creación de APIs para que las aplicaciones se comuniquen.

Tecnología	Lenguaje	Uso Principal	Ejemplos de Frameworks / Librerías
Java con Spring Boot	Java	APIs RESTful, microservicios, aplicaciones web de alta escala y enterprise	Spring MVC, Spring WebFlux, Spring Data REST, Hibernate

Notas rápidas:

- La elección depende del equipo, del ecosistema y de los requisitos no funcionales (latencia, concurrencia, facilidad de despliegue, integraciones).
- Para APIs públicas y rápidas de poner en marcha, frameworks como Express, Flask o Laravel son habituales.
- Para sistemas de gran escala o enterprise, Spring Boot y ASP.NET Core ofrecen herramientas maduras para seguridad, transacciones y despliegue.

3.7 Gestores de Bases de Datos

Los gestores de bases de datos son componentes fundamentales en cualquier plataforma web moderna. Son software encargados de almacenar, estructurar y recuperar grandes volúmenes de datos de manera eficiente.

- **MySQL / MariaDB**
Gestores de bases de datos relacionales de código abierto, muy populares por su eficiencia y velocidad. Se usan frecuentemente junto a PHP y otras pilas LAMP. MariaDB es un fork de MySQL totalmente libre.
- **PostgreSQL**
Gestor relacional de código abierto conocido por su robustez, cumplimiento de estándares y por ofrecer funcionalidades avanzadas para consultas complejas y control de integridad.
- **SQL Server**
Sistema gestor de bases de datos de Microsoft, habitual en entornos empresariales Windows (WISA). Ofrece buena integración con el ecosistema Microsoft y herramientas empresariales.

- **MongoDB**

Base de datos NoSQL orientada a documentos. Es flexible en el esquema, escalable y adecuada para datos semi-estructurados o aplicaciones que requieren cambios frecuentes en el modelo de datos.

3.9 Servidor Web y Servidor de Aplicaciones

Servidor Web

Software especializado en recibir peticiones HTTP y devolver recursos solicitados. Se enfoca principalmente en servir contenido estático como HTML, CSS, imágenes y archivos JavaScript.

Ejemplos: Apache HTTP Server, Nginx

Funciones principales:

- Atender y gestionar peticiones HTTP
- Servir contenido estático (archivos)
- Administrar múltiples sitios mediante Virtual Hosts
- Aplicar reglas de seguridad, compresión y redirecciones
- Reenviar peticiones dinámicas a aplicaciones externas
- Funcionar como reverse proxy para balanceo de carga y seguridad

Servidor de Aplicaciones

Software diseñado para ejecutar la lógica de negocio de una aplicación. No solo entrega archivos, sino que procesa código, realiza operaciones complejas y genera contenido dinámico personalizado.

Ejemplo: Apache Tomcat (para Java)

Funciones principales:

- Ejecutar código del lado del servidor
- Generar contenido dinámico según la petición
- Gestionar sesiones de usuario y autenticación
- Conectarse y consultar bases de datos
- Proporcionar entorno de ejecución para aplicaciones

3.10. Despliegue de Aplicaciones Web

Concepto de Despliegue

Proceso de poner una aplicación web en producción para que sea accesible a usuarios finales. Requiere tres componentes fundamentales:

Software

Conjunto de componentes tecnológicos que ejecutan la aplicación:

- **Sistema operativo** (Linux, Windows Server)
- **Servidor web** (Apache, Nginx) - sirve contenido estático
- **Servidor de aplicaciones** (Tomcat, Node.js) - ejecuta lógica de negocio
- **Runtime del lenguaje** (JDK para Java, PHP interpreter, .NET runtime)
- **Gestor de bases de datos** (MySQL, PostgreSQL, MongoDB)
- **Herramientas de desarrollo**: Docker Desktop, XAMPP (para entornos locales)

Hardware

Infraestructura física o virtual que soporta la aplicación:

- **Servidor** con capacidad de CPU, RAM y almacenamiento adecuada
- **Escalabilidad** para manejar carga de usuarios variable
- **Recursos ajustables** según demanda y crecimiento

Dependencias

Componentes externos necesarios para el funcionamiento:

- **Librerías específicas** del framework o proyecto
- **Paquetes**: archivos JAR (Java), NuGet (C#), Composer (PHP)
- **Módulos y extensiones** requeridos por la aplicación

Escalabilidad (Vertical y Horizontal, Clusters y Balanceadores de Carga)

La escalabilidad es la capacidad de un sistema web para absorber un aumento de carga sin perder rendimiento ni funcionalidad. Resulta esencial para cualquier aplicación moderna.

Escalabilidad vertical ("Scale Up"): Implica aumentar los recursos (CPU, RAM, almacenamiento) del servidor existente.

Escalabilidad horizontal ("Scale Out"): Consiste en añadir más servidores o “nodos” y repartir la carga entre ellos.

Clusters: Conjunto de servidores que trabajan de forma coordinada como una única unidad lógica, aumentando la disponibilidad y el rendimiento.

Balanceadores de carga: Dispositivos o aplicaciones que distribuyen el tráfico entre varios servidores. Optimizan los recursos, reducen tiempos de respuesta y evitan la saturación de un solo nodo.

Despliegue en Contenedores (Docker, Kubernetes)

Docker: Plataforma de contenedores que facilita la creación y ejecución de aplicaciones mediante “imágenes”. Los contenedores representan una forma ligera de virtualización, más eficiente que las máquinas virtuales tradicionales. El proceso incluye crear Dockerfile(s), generar imágenes, subirlas a un registro (como Docker Hub) y desplegarlas.

Docker Compose: Herramienta que permite orquestar varios contenedores a la vez y ajustar la configuración de ellos.

Kubernetes: Plataforma de orquestación que automatiza el despliegue, escalado y gestión de contenedores. Permite definir autoescalado, balanceo de carga y configurar réplicas para garantizar alta disponibilidad.

Despliegue en la Nube

Proveedores principales:

- AWS (Amazon)
- Google Cloud Platform (GCP)
- Microsoft Azure

Proceso de despliegue:

1. **Configurar infraestructura:** Instancias, bases de datos, redes
2. **Implementar código:** Usar PaaS (Elastic Beanstalk, App Engine)
3. **Configurar escalado:** Auto-scaling + load balancers
4. **Asegurar:** Firewalls, políticas, SSL/TLS
5. **Probar y validar**
6. **Monitorizar:** Logs, métricas, alertas
7. **Documentar** configuraciones y procedimientos

Estrategias de Despliegue

Despliegue Estándar

¿Qué es?

- Parar toda la aplicación → Reemplazar versión → Reiniciar
- **Downtime inevitable**

Cuándo usarlo:

- Aplicaciones pequeñas o internas
- Cuando minutos/horas de interrupción no son críticos

Rolling Updates

¿Qué es?

- Actualización gradual servidor por servidor
- Versiones antigua y nueva conviven temporalmente

Cuándo usarlo:

- Necesidad de disponibilidad 24/7
- Versiones compatibles entre sí

Blue/Green

¿Qué es?

- Dos entornos idénticos (Azul = producción, Verde = nuevo)
- **Switch instantáneo** entre entornos
- **Rollback inmediato** si hay problemas

Cuándo usarlo:

- Máxima seguridad en despliegues
- Necesidad de reversión rápida

Canary Releases

¿Qué es?

- Nueva versión a **pequeño grupo de usuarios** (1-5%)
- **Expansión gradual** si todo va bien

Cuándo usarlo:

- Validar funcionalidades críticas en producción
- Detectar problemas sin afectar a todos los usuarios

Feature Flags

¿Qué es?

- Interruptores en código para activar/desactivar funciones
- Sin nuevo despliegue necesario

Cuándo usarlo:

- Lanzamientos controlados por segmentos
- "Apagado de emergencia" de funcionalidades

CI/CD - Integración y Despliegue Continuo

Integración Continua (CI)

¿Qué hace?

- Los desarrolladores integran código frecuentemente en un repositorio compartido
- Cada commit activa automáticamente:
 - Build del proyecto
 - Ejecución de pruebas (unitarias, integración)

Objetivo:

- Detectar errores inmediatamente
- Mantener el código siempre estable

Entrega Continua (CD-Continuous Delivery): Extiende la CI garantizando que el código validado esté siempre listo para desplegar en producción. Incluye pruebas adicionales y despliegues en entornos de staging, normalmente con una aprobación manual previa.

